ACTL3142 Tutorial

Week 9: Tree-Based Methods

Nick Guo

School of Risk and Actuarial Studies, UNSW Sydney

T2 2026

Recap: Everything So Far Has Been Linear in β

- **Week 3 (MLR)**: $y = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p + \varepsilon$, linear in the predictors *and* the coefficients.
- **Week 7 (Ridge/Lasso)**: the same model with a penalty on $\|\beta\|$, tuned by cross-validation.
- **Week 8 (Splines, GAMs)**: $y = \beta_0 + \sum_k \beta_k \, b_k(x) + \varepsilon$. The fitted curve bends, but x is replaced by **fixed** transformations of x: still one global equation, still linear in β.

A tree has **no β and no global equation**.

A tree is a learned basis expansion

A tree is $\hat{f}(\mathbf{x}) = \sum_{m=1}^{J} c_m \, I(\mathbf{x} \in R_m)$, the same form as Week 8 except that the regions R_m are **estimated from the data**. This yields interactions automatically, at the cost of instability in a single tree.

Decision Trees: The Idea

A decision tree splits the predictor space into **non-overlapping rectangular regions** $R_1, \ldots, R_J$ using a sequence of binary rules.

Regression: predict $\hat{y} = \bar{y}_{R_j}$, the mean of the training observations in that region.
Classification: predict the **most common class** in the region.

Advantages:

- Easy to interpret and visualise
- Capture interactions and non-linearities without being specified in advance
- Accept qualitative predictors directly, with no assumption on the form of $f(X)$

Limitation

A single tree has **high variance**: a small change in the training data can produce an entirely different tree. This motivates pruning and, subsequently, ensemble methods.

Growing a Regression Tree: Recursive Binary Splitting

A **loss function** determines where to split. For regression trees this is **squared error**, with each region predicting $\hat{y}_{R_j} = \bar{y}_{R_j}$.

Searching over all possible partitions is computationally infeasible, so the algorithm is **greedy and top-down**:

1. Consider all predictors X_j and all cutpoints s
2. Select the (j, s) minimising the combined RSS of the two child nodes:

$$\sum_{i:\, \mathbf{x}_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:\, \mathbf{x}_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2$$

   where $R_1(j, s) = \{X \mid X_j < s\}$ and $R_2(j, s) = \{X \mid X_j \geq s\}$
3. Repeat within **one of the two new regions**, until a stopping rule is met (e.g. each leaf contains ≤ 5 observations)

"Greedy" means each split is only locally optimal: the algorithm never checks whether a worse split now would permit a better tree later. The same principle as forward stepwise selection.

Classification Trees: Loss Functions

RSS no longer applies, so the loss instead measures **node impurity**. Let $\hat{p}_{mk}$ be the proportion of class k in node m:

Loss function	**Formula**	**Use it for**
Classification error	$1 - \max_k(\hat{p}_{mk})$	Pruning, reporting
Gini index	$\sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$	Growing
Entropy	$-\sum_{k=1}^{K} \hat{p}_{mk} \ln(\hat{p}_{mk})$	Growing

All three equal zero at a pure node and are maximised when the classes are evenly mixed. Each is a **per-observation** loss, so a split is scored by weighting each child by the number of observations it receives:

$$\Delta \;=\; Q(\text{parent}) \;-\; \tfrac{n_L}{n}\, Q(L) \;-\; \tfrac{n_R}{n}\, Q(R),$$

and the chosen (j, s) is the one maximising Δ. Regression conceals this step, since RSS is already summed over observations and therefore weights by node size implicitly.

Why not grow the tree using classification error?

It depends only on the majority class, so a split can make both children purer while leaving it unchanged. Gini and entropy respond to *any* change in the class proportions and can therefore still rank such splits

Cost-Complexity Pruning: The Criterion

A fully grown tree overfits. Rather than stopping early, **grow a large tree T_0 and then prune it back**: a weak split may permit a much stronger split beneath it, which early stopping would never reach.

Add a **penalty on tree size**:

$$C_\alpha(T) = \underbrace{\sum_{m=1}^{|T|} \sum_{i:\, \mathbf{x}_i \in R_m} (y_i - \hat{y}_{R_m})^2}_{\text{fit, } = R(T)} + \underbrace{\alpha\,|T|}_{\text{complexity}}$$

$|T|$ is the number of leaves and $\alpha \geq 0$ is the tuning parameter: $\alpha = 0$ returns T_0, while large α collapses the tree toward its root.

How the pruning actually proceeds

α is a price per leaf. Pruning removes a **whole branch at a time**, not one leaf: as α rises, the internal node whose subtree buys the smallest RSS reduction *per leaf* is collapsed into a single leaf. Repeating this gives a **nested** sequence $T_0 \supset T_1 \supset \cdots \supset \{\text{root}\}$, each tree optimal over an interval of α, so cross-validation chooses among at most $|T_0|$ candidates.

Structurally identical to ridge and lasso, with complexity measured in leaves.

Choosing α by Cross-Validation

Each row of `rpart`'s `cptable` is one tree in that nested sequence. From the lecture's `rpart(log(Salary) ~ Years + Hits)` fit on `Hitters`:

CP	nsplit	$\lvert T \rvert$	rel error	xerror	xstd
0.4446	0	1	1.0000	1.0072	0.0657
0.1145	1	2	0.5554	0.5640	0.0590
0.0445	2	3	0.4409	**0.4620**	0.0575
0.0183	3	4	0.3964	**0.4290**	0.0594
0.0169	4	5	0.3781	0.4330	0.0639
0.0111	5	6	0.3612	0.4329	0.0660
0.0100	6	7	0.3501	0.4445	0.0662

`rel error` is the training error, which decreases monotonically. `xerror` is the **cross-validated** error, which does not.

- **Minimum** `xerror` is 0.4290 at 3 splits, giving **4 leaves**.
- **1-SE rule**: the smallest tree within one `xstd` of the minimum. Here $0.4290 + 0.0594 = 0.4884$, which 2 splits satisfies, giving **3 leaves**. This is the tree the lecture prunes to, via `cptable[3, "CP"]`.

`rpart`'s `cp` is $\alpha / R(\text{root})$, rescaled to $[0, 1]$. The table terminates at the default `cp` = 0.01, so larger trees are never fitted.

Why Ensemble Methods?

A single decision tree is:

- \+ Interpretable and intuitive
- – High variance: small changes in the data produce a very different tree
- – Often uncompetitive in predictive accuracy

Key insight: averaging many high-variance, low-bias models **reduces variance** while leaving bias low.

If $Z_1, \ldots, Z_B$ are i.i.d. with variance σ^2, then $\text{Var}(\bar{Z}) = \sigma^2/B$.

Three ensemble approaches:

- **Bagging**: deep trees fitted to bootstrap samples and averaged (variance reduction)
- **Random forests**: bagging with the predictors available at each split restricted at random, decorrelating the trees
- **Boosting**: small trees grown sequentially, each correcting the ensemble's remaining error (bias reduction)

Bagging (Bootstrap Aggregation)

Idea: many independent training sets are unavailable, but they can be approximated by **bootstrapping**.

Procedure:

1. Draw B bootstrap samples (n observations, **with replacement**)
2. Fit a **deep, unpruned** tree to each
3. Aggregate: regression $\hat{f}_{\text{bag}}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(\mathbf{x})$; classification, majority vote

Why not prune the individual trees?

Deep trees have low bias and high variance, and averaging is what eliminates the variance. Each tree is therefore left as flexible as possible.

Increasing B does not cause overfitting: additional trees only reduce variance further, with diminishing returns. B is not a tuning parameter, and should simply be set large enough.

Out-of-Bag (OOB) Error

Bootstrapping supplies a validation set at no additional cost. The probability that observation i is excluded from a given bootstrap sample is

$$\left(1 - \tfrac{1}{n}\right)^n \longrightarrow e^{-1} \approx 0.368,$$

so each tree is fitted on roughly two-thirds of the data, leaving the remainder **out of bag**.

The procedure:

1. For observation i, identify the trees for which i was OOB (approximately $B/3$)
2. Average their predictions, or take their majority vote, giving $\hat{y}_i^{\text{OOB}}$
3. OOB error $= \frac{1}{n}\sum_i \left(y_i - \hat{y}_i^{\text{OOB}}\right)^2$, or the misclassification rate

The estimate is valid because no tree used in step 2 was fitted using y_i, and it requires **one** fit where k-fold CV requires k.

Mildly pessimistic, since each $\hat{y}_i^{OOB}$ uses only $\sim B/3$ trees and a smaller ensemble predicts less well. "With B sufficiently large, OOB error is virtually equivalent to LOOCV" (James et al., 2021).

Random Forests

Problem with bagging: if one predictor is very strong, *every* bootstrapped tree splits on it first. The trees are then highly **correlated**, and averaging correlated quantities reduces variance far less effectively.

The remedy: at each split, consider only a **fresh random subset** of m of the p predictors.

- Typical choice: $m \approx \sqrt{p}$ (classification), $m \approx p/3$ (regression)
- The strong predictor is unavailable at most splits, so weaker predictors are used more often
- Variance falls, at the cost of some bias in each individual tree
- When $m = p$, a random forest **is** bagging

Why decorrelation helps

With pairwise correlation ρ between trees, the variance of their average is $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. Only the second term vanishes as B grows, so for large B the first term is **all that remains**. Reducing ρ is the only way to reduce it.

Variable Importance

The single-tree interpretation is lost: hundreds of trees, each using different predictors and leaves. The predictors can nevertheless still be ranked, in two quite different ways.

1. Loss reduction (`IncNodePurity`): for each predictor, sum the reduction in RSS (regression) or Gini index (classification) over every split using it, averaged over the B trees. Obtained directly from the fit, but biased toward continuous and many-levelled predictors, which are offered more candidate splits.

2. Permutation importance (`%IncMSE`): permute predictor j in the OOB data and record the increase in OOB error. If the model relied on j, disrupting it degrades performance. Measured on held-out data, so negative values (as in the lecture's flood data) simply indicate no signal.

In R: `importance(rf)` returns both columns (requires `importance = TRUE` when fitting), and `summary(gbm_fit)` returns boosting's "relative influence".

Interpret with care when predictors are correlated: they share the credit, so a useful variable can appear unimportant. This measures predictive contribution, not causal effect.

Boosting: The Idea

Bagging and random forests build B trees **in parallel** on resampled data and then average them. Boosting builds **one** model, **incrementally**:

$$\hat{f}(\mathbf{x}) = \sum_{b=1}^{B} \lambda \hat{f}^b(\mathbf{x}),$$

where each $\hat{f}^b$ is a small correction. After b trees, the ensemble's remaining error (the **residuals** r_i) becomes the target of the next tree. There is no bootstrapping: every tree uses all the data, with a new response each round.

Why deliberately weak learners?

A *deep* tree fitted to the residuals in round 1 drives them to nearly zero immediately, leaving a single overfit tree and no work for rounds 2 to B. Boosting requires every step to be constrained: few splits (d small) and a small step size (λ small). This is "slow learning".

Boosting: The Algorithm (Regression Trees)

1. Set $\hat{f}(\mathbf{x}) = 0$ and $r_i = y_i$ for all i.
2. For $b = 1, 2, \ldots, B$:
   (a) Fit a tree $\hat{f}^b$ with d splits (so $d + 1$ leaves) to the data $(X, \mathbf{r})$
   (b) Add a **shrunken** version of it: $\hat{f}(\mathbf{x}) \leftarrow \hat{f}(\mathbf{x}) + \lambda \hat{f}^b(\mathbf{x})$
   (c) Update the residuals: $r_i \leftarrow r_i - \lambda \hat{f}^b(\mathbf{x}_i)$
3. Output $\hat{f}(\mathbf{x}) = \sum_{b=1}^{B} \lambda \hat{f}^b(\mathbf{x})$.

The response in step (a) is $\mathbf{r}$, not $\mathbf{y}$. After round 1 the trees never use the original response again.

One round with a stump ($d = 1$): find the single split, over all predictors and cutpoints, best explaining the *current* residuals; predict the mean residual in each of its two leaves; then move a fraction λ of the way there.

The greedy split search is identical to an ordinary tree. Only the response being fitted changes.

Boosting: A Quick Example

Four observations, stumps ($d = 1$), learning rate $\lambda = 0.5$.

	$x = 1$	$x = 2$	$x = 3$	$x = 4$
y	0	0	12	24
Round 1: residuals $\mathbf{r}$	0	0	12	24
best stump is $x < 3$, giving $\hat{f}^1$	0	0	18	18
$\hat{f} \leftarrow \hat{f} + 0.5\,\hat{f}^1$	0	0	9	9
Round 2: residuals $\mathbf{r}$	0	0	3	15
best stump is $x < 4$, giving $\hat{f}^2$	1	1	1	15
$\hat{f} \leftarrow \hat{f} + 0.5\,\hat{f}^2$	0.5	0.5	9.5	16.5

Round 1 permits one cut, so $x = 3$ and $x = 4$ must share a leaf at 18. Round 2 fits the *residuals* and recovers the split round 1 could not make: two stumps produce a **three-region** step function.

Is fitting residuals equivalent to fitting a gradient?

Yes. With squared-error loss $L = \frac{1}{2}(y - \hat{f})^2$ we have $\partial L / \partial \hat{f} = -(y - \hat{f})$, so the residual **is** the negative gradient and step (b) is a gradient-descent step of size λ. Other losses replace $\mathbf{r}$ with their own negative gradient, hence **gradient boosting**.

Boosting: Two More Rounds

Every round follows the **same** procedure: grow a one-split tree using the current **r** as the response, take the mean residual in each leaf, and add λ times it.

	$x = 1$	$x = 2$	$x = 3$	$x = 4$
Round 3: residuals **r**	-0.5	-0.5	2.5	7.5
best stump is $x < 4$, giving $\hat{f}^3$	0.5	0.5	0.5	7.5
$\hat{f} \leftarrow \hat{f} + 0.5\,\hat{f}^3$	0.75	0.75	9.75	20.25
Round 4: residuals **r**	-0.75	-0.75	2.25	3.75
best stump is $x < 3$, giving $\hat{f}^4$	-0.75	-0.75	3	3
$\hat{f} \leftarrow \hat{f} + 0.5\,\hat{f}^4$	0.375	0.375	11.25	21.75
Target y, for comparison	0	0	12	24

Training RSS falls monotonically: $720 \to 234 \to 63 \to 20.25 \to 5.91$.

Round 3 worsened $x = 1, 2$

A single cut forces $\{-0.5, -0.5, 2.5\}$ into one leaf averaging $+0.5$, moving those two points *further* from zero. Total RSS still fell, since the gain at $x = 4$ outweighed the loss, and round 4 corrected them. A weak learner need not improve every observation, only the loss overall.

Boosting: The Three Tuning Parameters

B, the number of trees. Boosting can overfit if B is too large. Select it by CV (`gbm.perf(..., method = "cv")`) or early stopping on a validation set.

Why B overfits here but not in bagging: in bagging every tree independently estimates the same target, so the average converges. In boosting every tree **alters the model**, *so complexity grows with B.*

λ, the shrinkage or learning rate. Typically 0.01 or 0.001. It trades off against B: halving λ roughly doubles the number of trees required.

d, the splits per tree (giving $d + 1$ leaves). $d = 1$ is the standard starting point; the lecture's flood insurance example used $d = 2$.

In `gbm` this argument is called `interaction.depth`, for the reason given on the next slide.

Choosing them in practice

B and λ are chosen jointly: fix a small λ, then use CV to determine the required number of trees.

In practice `XGBoost` and `LightGBM` are the standard implementations rather than `gbm`: each tree is fitted to the **negative** *gradient of a chosen loss, with additional regularisation. Not examinable in this course.*

Interaction Depth: Why $d = 1$ Gives a GAM

A stump splits on **one** variable, so each $\hat{f}^b$ is a function of that variable alone. Grouping the B stumps by the variable they split on:

$$\hat{f}(\mathbf{x}) = \sum_{b=1}^{B} \lambda \hat{f}^b(\mathbf{x}) = \sum_{j=1}^{p} g_j(x_j), \qquad g_j(x_j) = \lambda \sum_{b\,:\,\text{splits on } x_j} \hat{f}^b(\mathbf{x}).$$

This is precisely the **GAM** from Week 8, $\hat{f} = \beta_0 + \sum_j g_j(x_j)$, except that each g_j is a step function built from shrunken stumps rather than a spline.

Why this caps interactions

No term in $\sum_j g_j(x_j)$ involves two variables simultaneously, so with $d = 1$ the effect of x_1 cannot depend on x_2, however large B becomes. With $d = 2$ a single tree can split on x_1 and then on x_2 beneath it, producing a region $\{x_1 < s_1\} \cap \{x_2 < s_2\}$: a genuine two-way interaction.

d therefore sets the maximum order of interaction the model can represent. Use $d = 1$ for an additive, interpretable fit, and increase it when predictors are expected to act jointly.

Bagging vs Random Forests vs Boosting

	Bagging	**Random Forest**	**Boosting**
Trees built	Independently	Independently	Sequentially
Fitted to	Bootstrap data	Bootstrap data	Current residuals
Individual trees	Deep (low bias)	Deep (low bias)	Small (stumps)
Improves by	Less variance	Less variance	Less bias
Predictors/split	All p	Random $m < p$	All p
B can overfit?	No	No	**Yes**
Free OOS error	OOB	OOB	No, use CV

All three sacrifice the single-tree interpretation for accuracy, and all three return variable importance in its place.

Boosting often attains the best predictive performance, but it is the only one of the three requiring careful tuning. Bagging and random forests perform well close to their default settings.

Thanks for coming! See you next week.